Back To Basics
Loops in C++

NICOLAI M. JOSUTTIS

Cppcon
The C++ Conference

2026 September 12 - 18

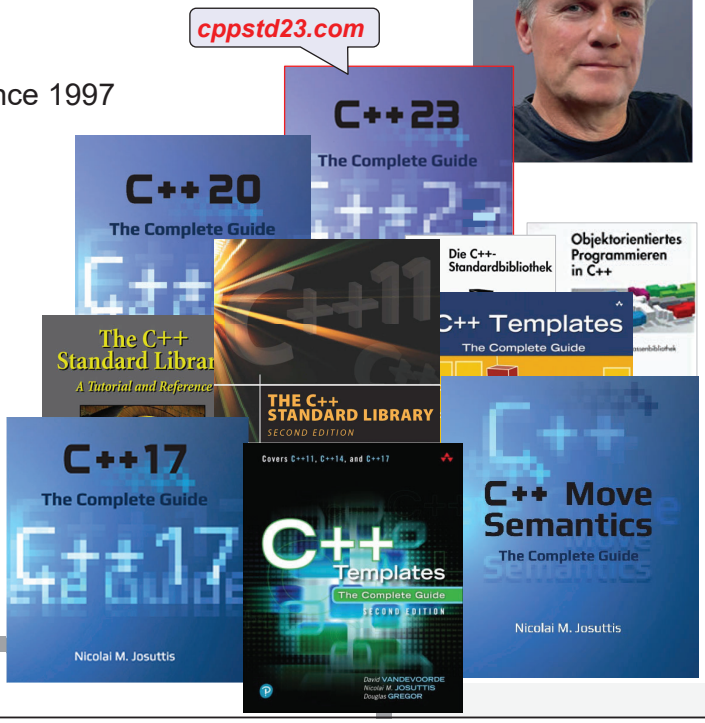
Josuttis C++

Nicolai M. Josuttis

- **Independent consultant**
  - Continuously learning since 1962
- **C++:**
  - since 1990
  - ISO Standard Committee since 1997
- **Other Topics:**
  - Systems Architect
  - Technical Manager
  - SOA
  - X and OSF/Motif







C++

©2026 by josuttis.com

C++

Basic Loops

©2026 by josuttis.com

Loops from C

C

`while (`*cond*`)` *statement(s)*

`do` *statement(s)* `while (`*cond*`)`

`for (`*init*`;` *cond*`;` *reinit*`)` *statement(s)*

```
while (!done) {
  stuff();
}
```

```
do {
  stuff();
}
while (!done);
```

```
for (int i = 0; i < 10; ++i) {
  print(i);
}
```

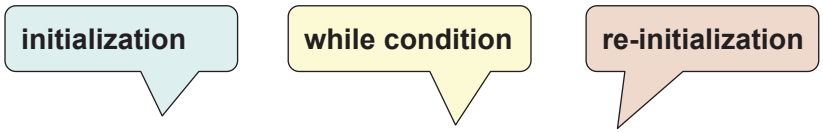
– *cond*:
  - Boolean expression
  - Integral value (`0` is `false`)

– *statement(s)*:
  - single statement with semicolon
  - block of statements with `{...}`

©2026 by josuttis.com

The for Loop

C / C++11




```
for (int i = 0, j = 10; i < j; ++i, --j) {
  std::cout << i << " and " << j << '\n;
}
```

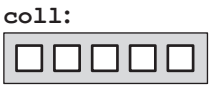
```
for (auto pos = coll.begin(); pos != coll.end(); ++pos) {
  std::cout << *pos << '\n';
}
```


©2026 by josuttis.com

Range-Based for Loop

C++11

- **Foreach loop**: *„do something with each element“*
  - Processing ranges from `begin()` to `end()`




```
std::vector<std::string> coll;
...
for (std::string elem : coll) {          // print all elements of coll
  process(elem);
}
```

Note: `elem` is a copy of the element in `coll`

```
for (const std::string& elem : coll) {   // print elems without copying
  process(elem);
}
```

`&` (reference) to avoid copying the element
`const` to ensure that it can't be modified

```
for (std::string& elem : coll) {         // modify each element
  modify(elem);
}
```

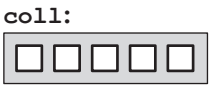
Without `&`, we would modify copies of elems

©2026 by josuttis.com

Range-Based `for` Loop

C++11

- **Foreach loop**: *„do something with each element“*
  - Processing ranges from `begin()` to `end()`




```
std::vector<std::string> coll;
...
```

```
for (auto elem : coll) {
  process(elem);
}
```

// print all elements of `coll`

Note: `elem` is a copy of the element in `coll`

```
for (const auto& elem : coll) {
  process(elem);
}
```

// print elems without copying

`&` (reference) to avoid copying the element
`const` to ensure that it can't be modified

```
for (auto& elem : coll) {
  modify(elem);
}
```

// modify each element

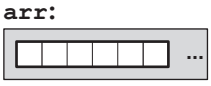
Without `&`, we would modify copies of elems

©2026 by josuttis.com

Range-Based `for` Loop

C++11

- **Foreach loop**: *„do something with each element“*
  - Processing ranges from `begin()` to `end()`
  - Also for raw arrays




```
std::string arr[100];
...
```

```
for (auto elem : arr) {
  process(elem);
}
```

// print all elements of `arr`

Note: `elem` is a copy of the element in `coll`

```
for (const auto& elem : arr) {
  process(elem);
}
```

// print elems without copying

`&` (reference) to avoid copying the element
`const` to ensure that it can't be modified

```
for (auto& elem : arr) {
  modify(elem);
}
```

// modify each element

Without `&`, we would modify copies of elems

©2026 by josuttis.com

Range-Based `for` Loop

C++11

- **The expression:**

```
for ( decl : rg ) {
  statement
}
```

```
// print all elements of coll:
for (const std::string& elem : coll) {
  std::cout << elem << '\n';
}
```

is roughly interpreted as:

```
for (auto pos = rg.begin(), end = rg.end(); pos != end; ++pos) {
  decl = *pos;
  statement
}
```

```
// print all elements of coll:
for (auto pos = coll.begin(), end = coll.end(); pos != end; ++pos) {
  const std::string& elem = *pos;
  std::cout << elem << '\n';
}
```

Without declaring `elem` as reference `elem` would be a copy of the element

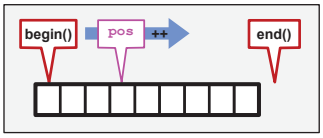
©2026 by josuttis.com

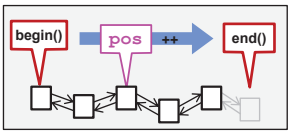
Iterating with Generic Code

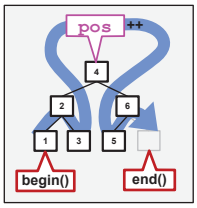
C++11

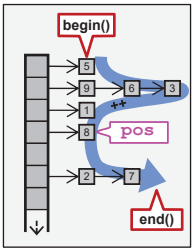
```
for (const auto& elem : range) {
  process(elem);
}
```

```
for (auto pos = range.begin(); pos != range.end(); ++pos) {
  process(*pos);
}
```







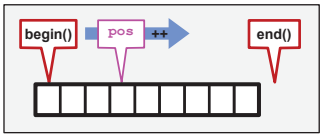


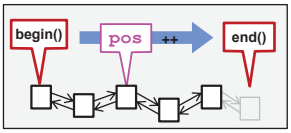
©2026 by josuttis.com

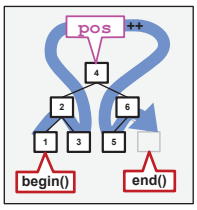
How the Range-Based `for` Loop is Specified

C++11

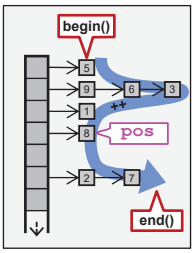
```
for (const auto& elem : range) {
  process(elem);
}
```








```
{
  auto&& rg = range;              // give a name for what we iterate over
  auto beg = rg.begin();          // call begin() only once
  auto end = rg.end();            // call end() only once
  for (auto pos = beg; pos != end; ++pos) {
    process(*pos);
  }
}
```

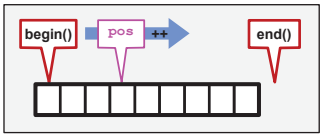


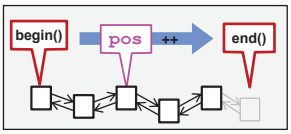
©2026 by josuttis.com

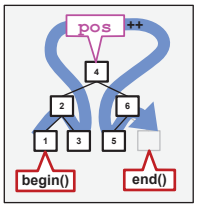
How the Range-Based `for` Loop is Specified

C++11

```
for (const auto& elem : getColl()) {
  process(elem);
}
```





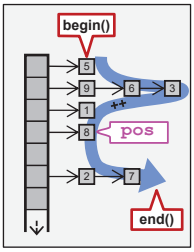



```
{
  auto&& rg = getColl();          // give a name for what we iterate over
  auto beg = rg.begin();          // call begin() only once
  auto end = rg.end();            // call end() only once
  for (auto pos = beg; pos != end; ++pos) {
    process(*pos);
  }
}
```

As `auto&&`, `rg` is a *universal / forwarding reference*

- Can refer to `const` and non-`const`
- Can refer to named object (lvalue) and temporary (rvalue)

`auto`	would copy or move
`const auto&`	would make the collection `const`
`auto&`	would not compile for rvalues (temporaries)



©2026 by josuttis.com

Range-Based `for` Loops with Initialization C++20

```
// print elements with leading position:                    C++11
int i = 1;
for (const auto& elem : coll) {
  std::cout << ++i << ": " << elem << '\n';
}
```

```
// print elements with leading position:                    C++20
for (int i = 1; const auto& elem : coll) {
  std::cout << ++i << ": " << elem << '\n';
}
```

`i` exists until the end of the loop

```
// print elements of coll while coll is locked:            C++20
for (std::lock_guard lg{collMx}; const auto& elem : coll) {
  std::cout << elem << '\n';
}
```


©2026 by josuttis.com

C++

Broken Range-Based `for` Loop

©2026 by josuttis.com

Broken Range-Based `for` Loop 1/2

C++11/C++17

```
std::string getString(int num)
{
  if (num <= 0) {
    return "";              // return no value
  }
  // return a test string with num characters:
  return std::string(num, '+');
}

int main()
{
  // OK:
  std::cout << getString(20) << '\n';

  // OK:
  for (const auto& elem : getString(20)) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
++++++++++++++++++++
+ + + + + + + + + + + + + + + + + + + +
```


©2026 by josuttis.com
https://www.godbolt.org/z/eqffjvbG8

Broken Range-Based `for` Loop 2/2

C++11/C++17

```
std::optional<std::string> getOptString(int num)
{
  if (num <= 0) {
    return std::nullopt;    // return no value
  }
  // return a test string with num characters:
  return std::string(num, '+');
}

int main()
{
  // OK:
  std::cout << getOptString(20).value() << '\n';

  // RUNTIME ERROR (undefined behavior) before C++23:
  for (const auto& elem : getOptString(20).value()) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Possible Output:

```
++++++++++++++++++++
□ □
{ □ □    u □ p + + + +
```


©2026 by josuttis.com
https://www.godbolt.org/z/zs8rdoscd

Broken Range-Based `for` Loop

C++11/C++14/C++17

Don't let the range-based `for` loop iterate over a reference to a temporary

- Otherwise: Fatal runtime error (**U**ndefined **B**ehavior)
- Fixed with C++23 (see http://wg21.link/p2012) VC++ fix with VS2026

- **Code broken before C++23:**

```
// iterate over an optional collection:
for (auto val : getOptionalColl().value()) ...      // Runtime ERROR before C++23

// iterate over a collection which is the first element in a vector:
for (auto elem : getVector()[0]) ...                // Runtime ERROR before C++23

// iterate over collection which is the first member in a tuple:
for (auto elem : std::get<0>(getTuple())) ...       // Runtime ERROR before C++23

// in general: iterate over a reference to a temporary:
for (auto elem : getTmp().getRef()) ...             // Runtime ERROR before C++23
```


©2026 by josuttis.com
https://www.godbolt.org/z/5caosGfW3

References to Temporaries

C++98/C++11

- **References extend the lifetime of return values**
- **No indirect lifetime extensions**
  - Extend only the lifetime to what they directly refer

```
const auto& rt1 = getTmp();                 // extends lifetime of return value

for (const auto& elem : rt1) {              // OK
  ...
}
```

```
const auto& rt2 = getTmp().getRef();        // extends lifetime of reference only
                                            // here, the temporary object returned by getTmp() is destroyed

for (const auto& elem : rt2) {              // RUNTIME ERROR
  ...
}
```


©2026 by josuttis.com

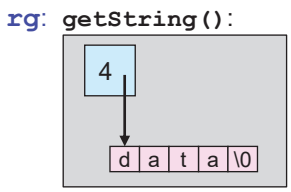
Range-Based `for` Loop Considered Harmful

C++11

- Range-based `for` loops use **references** to call `begin()` *and* `end()`

```
std::string getString()
{
  return "data";
}

for (char c : getString()) {
  std::cout << c << '\n';
}
```



Range-based `for` loop as specified:

```
auto&& rg = getString();        // rg extends the lifetime of the return value of getString()
```


©2026 by josuttis.com

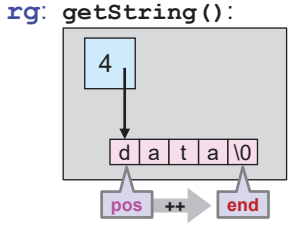
Range-Based `for` Loop Considered Harmful

C++11

- Range-based `for` loops use **references** to call `begin()` *and* `end()`

```
std::string getString()
{
  return "data";
}

for (char c : getString()) {
  std::cout << c << '\n';
}
```



Range-based `for` loop as specified:

```
auto&& rg = getString();        // rg extends the lifetime of the return value of getString()
// here, the temporary string returned by getString() is still alive
auto beg = rg.begin();
auto end = rg.end()
for (auto pos = beg; pos != end; ++pos) {
  char c = *pos;
  std::cout << c << '\n';
}
```


©2026 by josuttis.com
https://www.godbolt.org/z/eqffjvbG8

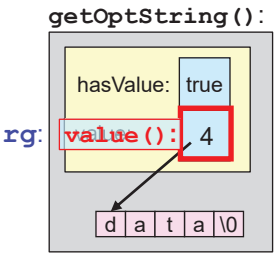
Range-Based `for` Loop Considered Harmful

C++11/C++17

- **Range-based `for` loops use references to call `begin()` *and* `end()`**

```
std::optional<std::string> getOptString()
{
  return "data";
}

for (char c : getOptString().value()) {
  std::cout << c << '\n';
}
```



Range-based `for` loop as specified:

```
auto&& rg = getOptString().value();      // rg extends only the lifetime of value()
```

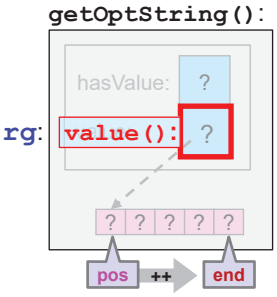

Range-Based `for` Loop Considered Harmful

C++11/C++17

- **Range-based `for` loops use references to call `begin()` *and* `end()`**
  => Severe **runtime errors** possible

```
std::optional<std::string> getOptString()
{
  return "data";
}

for (char c : getOptString().value()) {
  std::cout << c << '\n';
}
```



Range-based `for` loop as specified:

```
auto&& rg = getOptString().value();      // rg extends only the lifetime of value()
// here, the optional returned by getOptString() is destroyed
auto beg = rg.begin();
auto end = rg.end()
for (auto pos = beg; pos != end; ++pos) {
  char c = *pos;
  std::cout << c << '\n';
}
```

Don't iterate over a reference to a temporary

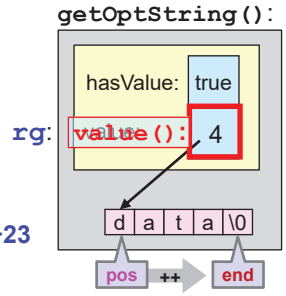
Range-Based `for` Loop Considered Harmful

C++23

- **Range-based `for` loops use references to call `begin()` *and* `end()`**
  => Severe **runtime errors** possible before C++23

```
std::optional<std::string> getOptString()
{
  return "data";
}

for (char c : getOptString().value()) { // OK since C++23
  std::cout << c << '\n';
}
```



Range-based `for` loop as specified:

```
auto&& rg = getOptString().value();     // rg extends only the lifetime of value()
// here, the optional returned by getOptString() is destroyed
auto beg = rg.begin();
auto end = rg.end()
for (auto pos = beg; pos != end; ++pos) {
  char c = *pos;
  std::cout << c << '\n';
}
```

Fixed with C++23:
- All temporaries for `rg` exist during the whole loop
- Feature test macro `__cpp_range_based_for` is `>= 202211`

Portable Use of the Range-Based `for` Loop

C++11/C++23

```
std::optional<std::string> getOptString(int num)
{
  if (num <= 0) {
    return std::nullopt;   // return no value
  }
  // return a test string with num characters:
  return std::string(num, '+');
}

int main()
{
  std::cout << getOptString(20).value() << '\n';

#if __cpp_range_based_for >= 202211
  for (const auto& elem : getOptString(20).value()) {
#else
  auto&& opt = getOptString(20);
  for (const auto& elem : opt.value()) {
#endif
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
++++++++++++++++++++
+ + + + + + + + + + + + + + + + + + + +
```

References to Temporaries (since C++23) C++23

- **C++23 fixes only the range-based `for` loop**
- **Still no indirect lifetime extensions**
  - References still extend only the lifetime to what they directly refer

```
for (const auto& elem : getTmp().getRef()) {     // OK since C++23
  ...
}
```

```
const auto& rt2 = getTmp().getRef();   // extends lifetime of reference only
                                       // here, still the temporary object
                                       // returned by getTmp() is destroyed
for (const auto& elem : rt2) {         // RUNTIME ERROR
  ...
}
```


©2026 by josuttis.com

C++

Loops on Ranges

©2026 by josuttis.com

Loops Over Ranges

C++98/C++20

- **On Ranges (containers, views, ...) you can iterate with:**
  - **Algorithms**
    - Loop over elements
    - Process each and every element
  - **Views** (since C++20)
    - Easy to skip certain elements
  - **Algorithms over Views**

©2026 by josuttis.com

Count Prime Numbers

C++

C++11

```
int num = 0;
for (auto elem : coll) {
  if (isPrime(elem)) {
    ++num;
  }
}
return num;
```

Performance (measured):
- No significant differences

C++98

```
return std::count_if(coll.begin(), coll.end(),
                     isPrime);
```

C++20

```
return std::ranges::count_if(coll,
                             isPrime);
```

C++20

```
int num = 0;
for (const auto& elem : coll | std::views::filter(isPrime)) {
  ++num;
}
return num;
```


©2026 by josuttis.com
https://www.godbolt.org/z/d1sG4M9q3

Copy Prime Numbers

C++

C++11

```
for (auto elem : coll) {
  if (isPrime(elem)) {
    dest.push_back(elem);
  }
}
```

```
std::copy_if(coll.begin(), coll.end(),
             std::back_inserter(dest),
             isPrime);
```

Performance (measured):

- `dest` is `list<>`:
  equally fast
- `dest` is `vector<>`:
  views take twice as long
  (with and without `reserve()`)

C++20

```
std::ranges::copy_if(coll,
                     std::back_inserter(dest),
                     isPrime);
```

C++23

```
dest.assign_range(coll | std::views::filter(isPrime));
```

C++23

```
dest = coll | std::views::filter(isPrime)
            | std::ranges::to<std::remove_cvref_t<decltype(dest)>>();
```


©2026 by josuttis.com
https://www.godbolt.org/z/K1Ev5acYW

Copy Every Second Element

C++

C++11

```
for (auto pos = coll.begin(); pos < coll.end(); pos += 2) {
  dest.push_back(*pos);
}
```

```
auto oddIndex = [index = 0](auto) mutable {
  return ++index % 2 == 1;
};
```

```
std::copy_if(coll.begin(), coll.end(),
             std::back_inserter(dest),
             oddIndex);
```

- Loop only compiles for random-access source ranges
- Tricky predicate (mutable)

Performance (measured):

- Views may be significant faster for vectors without `reserve()`

C++20

```
std::ranges::copy_if(coll,
                     std::back_inserter(dest),
                     oddIndex);
```

C++23

```
dest.assign_range(coll | std::views::stride(2));
```

C++23

```
dest = coll | std::views::stride(2)
            | std::ranges::to<std::remove_cvref_t<decltype(dest)>>();
```


©2026 by josuttis.com
https://www.godbolt.org/z/vKvhx67xW

Parallel Processing with Views

C++23

```
void processParallel(auto& coll, auto op)   // call op for all elements
{
  auto foreach = [] (auto& subView, auto op) {
    for (auto&& elem : subView) {
      op(elem);
    }
  };

  auto v1 = coll | std::views::stride(3);   // every third elem
  auto v2 = coll | std::views::drop(1)      // every third elem from 2nd
                 | std::views::stride(3);
  auto v3 = coll | std::views::drop(2)      // every third elem from 3rd
                 | std::views::stride(3);

  // process data in 3 parallel threads:
  std::jthread t1{foreach, v1, op};         // call op for every third elem
  std::jthread t2{foreach, v2, op};         // call op for every third elem from 2nd
  std::jthread t3{foreach, v3, op};         // call op for every third elem from 3rd

  t1.join();
  t2.join();
  t3.join();
}
```

generic helper to call op for each element

```
auto square = [] (auto& elem) {
  elem *= elem;
};
processParallel(data, square);
```


Parallel Processing with Views

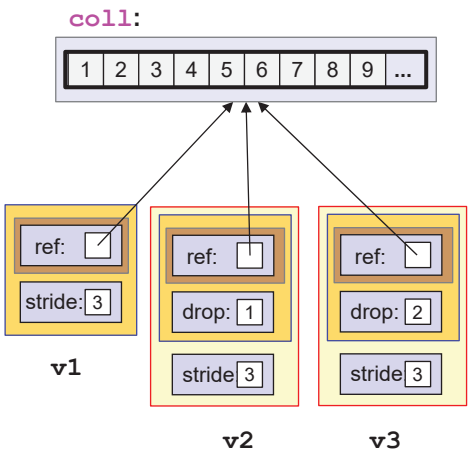
C++23

```
void processParallel(auto& coll, auto op)
{
  auto foreach = [] (auto& subView, auto op) {
    for (auto&& elem : subView) {
      op(elem);
    }
  };

  auto v1 = coll | std::views::stride(3);
  auto v2 = coll | std::views::drop(1)
                 | std::views::stride(3);
  auto v3 = coll | std::views::drop(2)
                 | std::views::stride(3);

  // process data in 3 parallel threads:
  std::jthread t1{foreach, v1, op};
  std::jthread t2{foreach, v2, op};
  std::jthread t3{foreach, v3, op};

  t1.join();
  t2.join();
  t3.join();
}
```



Summary

- C++ has several ways to loop
- The range-based `for` loop is tricky
- To iterate over elements
  - Use algorithms
  - Use views
- When which is best depends

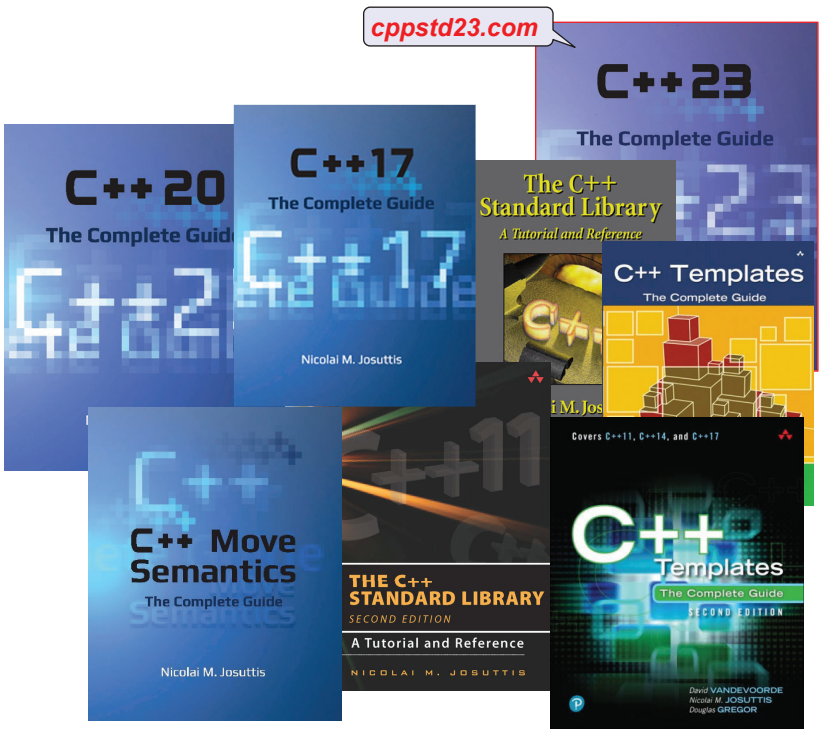
©2026 by josuttis.com

Thank You!



Nicolai M. Josuttis

www.josuttis.com
nico@josuttis.com
@NicoJosuttis



©2026 by josuttis.com